

自然语言处理上的对抗式攻击（下）：生成式攻击与防御

Auto-Encoder Attacks, Gumbel-Softmax, Robust Training, ASCC, and Detection



基于李宏毅老师《机器学习 2025》课程内容整理
2026 年 6 月 9 日

视频信息

频道：李宏毅深度学习课堂 (Bilibili)

时长：约 1 小时 10 分 19 秒

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=42>

目录

1	从词替换到生成式攻击	3
1.1	Generator、classifier 与攻击目标	3
1.2	重建、语义与攻击 loss	4
1.3	攻击与防御的交替视角	4
1.4	本章小结	5
2	离散采样不可微与 Gumbel-Softmax	5
2.1	为什么 argmax 会阻断梯度	6
2.2	Gumbel-Max trick	6
2.3	Softmax temperature relaxation	7
2.4	近似 one-hot 与梯度回传	9
2.5	本章小结	10
3	训练一个执行扰动的 Agent	10
3.1	动作空间	11
3.2	强化学习奖励	12
3.3	与 greedy 搜索的关系	13
3.4	本章小结	13
4	防御一：Adversarial Training	13
4.1	标准训练流程	13
4.2	文本 adversarial training 的局限	14
4.3	本章小结	14
5	防御二：Embedding 空间与 ASCC	14
5.1	Embedding ball adversarial training	15
5.2	ASCC 的动机	15
5.3	稀疏性与最坏组合	17
5.4	用不鲁棒分类器做数据增强	17
5.5	本章小结	18
6	防御三：推理期检测与修复	18
6.1	DISP：三段式检测与恢复	19
6.2	Embedding estimator	20
6.3	训练与推理流程	22
6.4	FGWS：用词频变化发现可疑替换	22
6.5	本章小结	23
7	总结与延伸	23

1 从词替换到生成式攻击

上一讲主要围绕 synonym substitution attack: 攻击者先找重要词, 再从同义词或语言模型候选中挑替换词。P42 继续讨论 evasion attack, 但重点从“逐词替换”转向“训练一个生成器”。如果一个 generator 能自动把原句 x 改成 x' , 同时让 x' 保持语义、语法自然并欺骗分类器, 那么攻击就可以被写成一个可训练的生成任务。

这条路线的动机很直接: 手写搜索规则会遇到大量启发式选择, 例如候选集合怎么建、先替换哪个词、约束阈值设多少。生成式方法希望让模型自己学会产生扰动句子, 再通过分类器 loss、重建 loss 和语义 loss 把它限制在合理范围内。

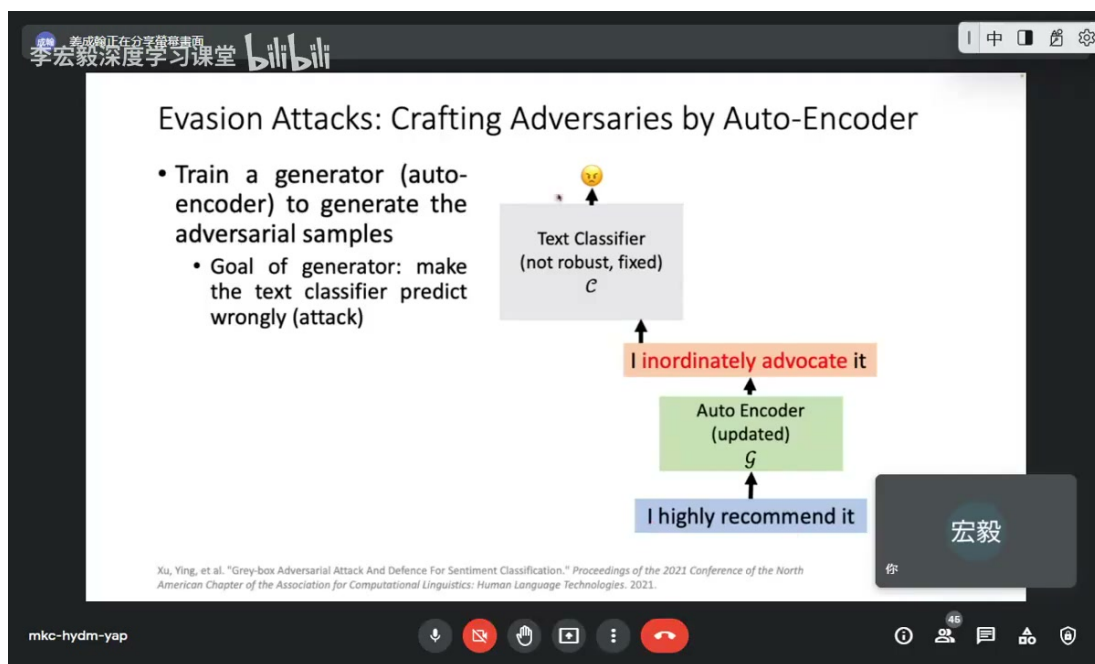


图 1: Auto-encoder attack 的基本目标: 训练 generator 产生 adversarial sample, 使固定的 text classifier 预测错误。¹

1.1 Generator、classifier 与攻击目标

设文本分类器为 C , 原始输入为 x , 真实标签为 y 。生成器或 auto-encoder 记为 G_θ , 它把 x 改写成:

$$x' = G_\theta(x).$$

攻击者希望 x' 同时满足两件事:

1. 对人类而言, x' 与 x 足够接近, 语义没有被偷换。
2. 对模型而言, $C(x')$ 不再输出 y , 或输出攻击者指定的目标类别 y_{tar} 。

对 untargeted attack, 可以让 adversarial loss 鼓励分类器降低真实标签概率:

$$L_{\text{adv}}(x') = \log p_C(y | x') \quad \text{或} \quad -L_C(C(x'), y),$$

具体符号取决于实现是最小化还是最大化。直观上, 生成器要让分类器对正确类别越来越没有信心。对 targeted attack, 则可以改成最小化:

$$L_{\text{tar}}(x') = -\log p_C(y_{\text{tar}} | x').$$

¹视频画面时间区间: 00:02:15–00:02:30。

1.2 重建、语义与攻击 loss

生成器不能只追求让分类器出错，否则它可能输出完全无关的句子。课程用 reconstruction、similarity 和 adversarial loss 共同约束生成过程。

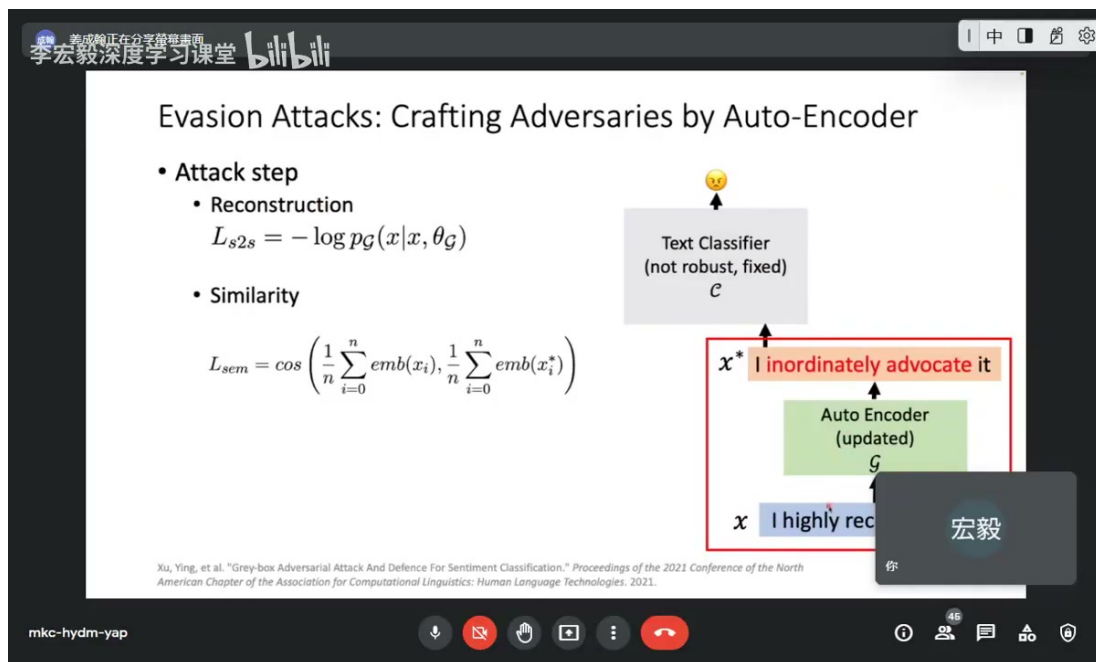


图 2: 攻击阶段的 loss 通常同时包含重建项、语义相似项和诱导分类器出错的 adversarial loss。²

可以把训练目标概括成：

$$\min_{\theta} \lambda_{\text{rec}} L_{\text{rec}}(x, G_{\theta}(x)) + \lambda_{\text{sim}} L_{\text{sim}}(x, G_{\theta}(x)) + \lambda_{\text{adv}} L_{\text{adv}}(G_{\theta}(x)).$$

其中：

L_{rec} 要求 auto-encoder 不要离原句太远，可理解为重建原句的语言建模损失。

L_{sim} 要求生成句与原句在 embedding 或句向量空间中接近。

L_{adv} 要求生成句让固定分类器犯错。

λ 控制三种目标的相对权重。

这些项之间天然冲突。若 L_{rec} 太强，生成器只会复读原句，攻击失败；若 L_{adv} 太强，生成器可能生成语义崩坏的句子；若 L_{sim} 设计得太弱，成功率会被低质量样本夸大。

1.3 攻击与防御的交替视角

同一个 auto-encoder 生成的样本也可以用于 defense。攻击阶段固定分类器、更新生成器；防御阶段可以固定或使用生成器产生的 adversarial examples，再更新分类器，让分类器学会把它们判回正确标签。

² 视频画面时间区间：00:05:30-00:05:45。

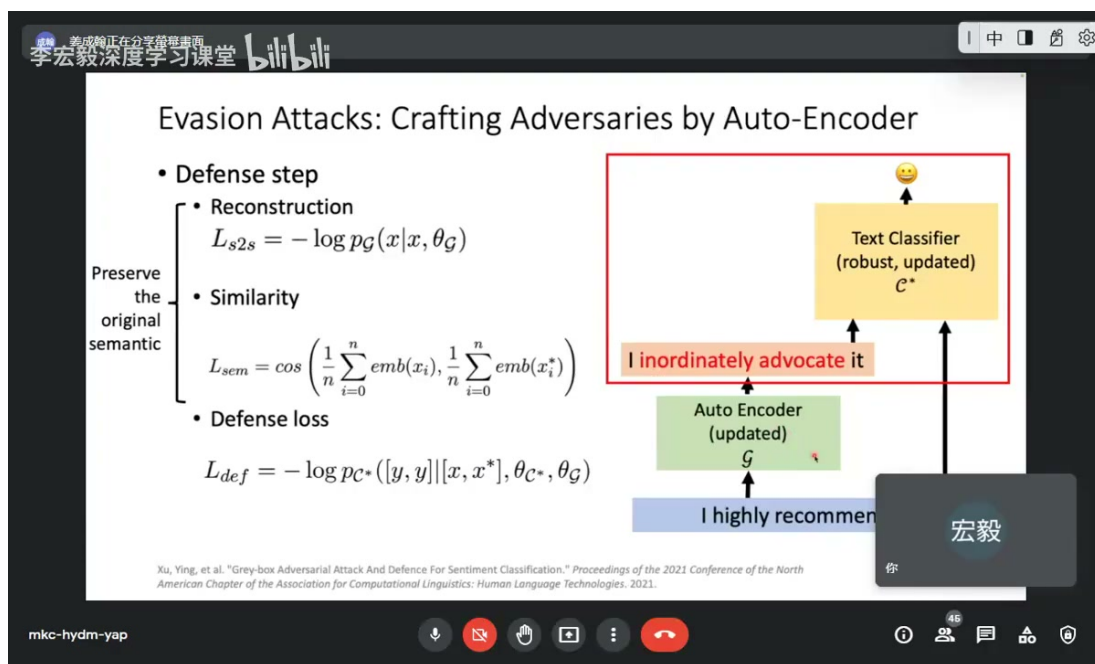


图 3: 防御阶段会把生成的 adversarial sample 喂回分类器，使分类器在这些扰动输入上仍预测正确。³

这对应到 adversarial training 的基本形式：

$$\min_{\phi} \mathbb{E}_{(x,y) \sim \mathcal{D}} [L_C(C_{\phi}(x), y) + \beta L_C(C_{\phi}(G_{\theta}(x)), y)].$$

第一项维持 clean input 的表现，第二项让模型在生成器产生的 x' 上也预测正确。这里的 G_{θ} 可以是攻击者训练出来的，也可以是防御者内部用来生成训练样本的攻击模块。

生成式攻击的核心张力

生成器越自由，越容易找到让分类器出错的输入；生成器越受约束，越能保证样本自然、语义一致，但攻击越困难。NLP adversarial generation 的难点，就是把“可攻击性”和“可读性”同时放进训练目标。

1.4 本章小结

Auto-encoder attack 把文本对抗样本生成看成是一个训练问题：generator 负责改写句子，classifier 提供攻击信号，reconstruction 和 similarity loss 保持文本质量。它比纯词替换搜索更“模型化”，但也引出了一个关键技术问题：文字是离散的，采样和 argmax 不可微，梯度如何从分类器传回 generator？

2 离散采样不可微与 Gumbel-Softmax

图像生成器输出连续像素，分类器 loss 可以直接反向传播到生成器。文本生成器不同：它通常输出词表上的概率分布，然后通过 sampling 或 argmax 选出一个 token，再查 embedding table 送入分类器。这个离散选择步骤不可微，导致分类器的梯度无法直接穿过采样步骤更新 auto-encoder。

³视频画面时间区间：00:08:45–00:09:00。

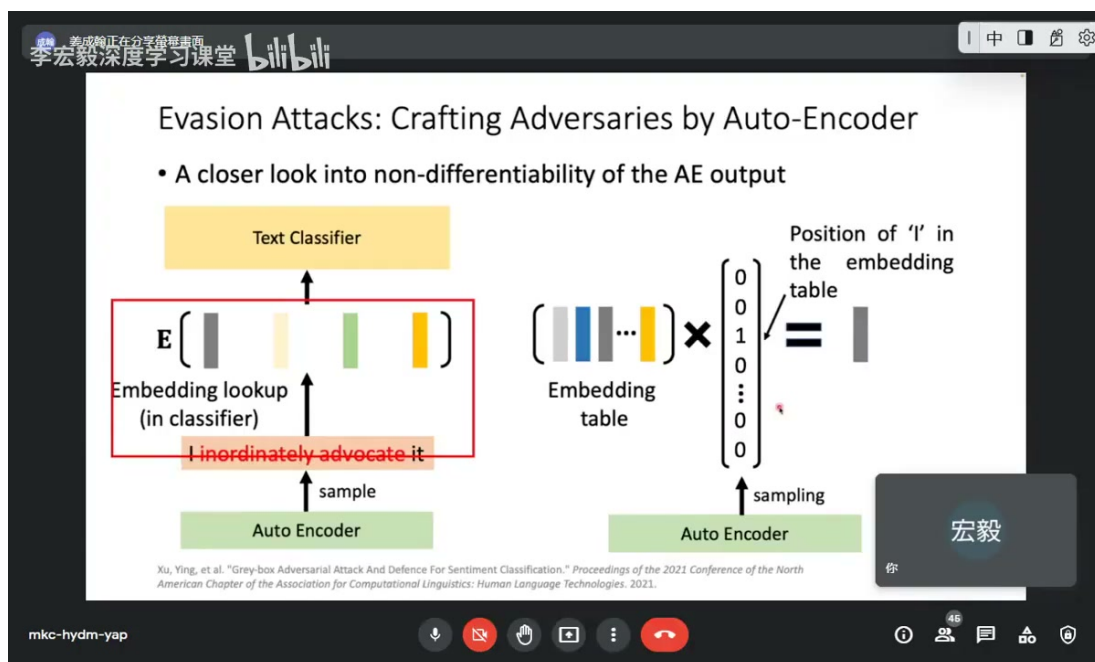


图 4: Auto-encoder 输出词表分布后若直接采样 token，再做 embedding lookup，采样位置会阻断梯度。⁴

2.1 为什么 argmax 会阻断梯度

假设生成器在某个位置输出 categorical distribution:

$$\pi = (\pi_1, \pi_2, \dots, \pi_V), \quad \sum_{i=1}^V \pi_i = 1.$$

离散生成通常取:

$$z = \text{onehot}(\arg \max_i \pi_i).$$

问题在于 $\arg \max$ 对输入概率的微小变化大多没有反应；一旦最大项没变，输出 one-hot 完全不变；最大项切换时又是突变。这样的函数几乎处处梯度为零，切换点不可导。若后面的 classifier loss 为 $L(C(Ez), y)$ ，梯度无法稳定回到 π 。

这也是 NLP 生成式攻击比图像生成式攻击麻烦的地方：模型真正想优化的是离散 token，但梯度优化喜欢连续空间。

2.2 Gumbel-Max trick

Gumbel-Max trick 提供了一种从 categorical distribution 中采样的写法。若 g_i 是独立同分布的 Gumbel noise:

$$g_i = -\log(-\log u_i), \quad u_i \sim \text{Uniform}(0, 1),$$

则可以用:

$$z = \text{onehot} \left(\arg \max_i (\log \pi_i + g_i) \right)$$

从 $\text{Cat}(\pi)$ 中采样。

⁴ 视频画面时间区间: 00:14:00-00:14:15。

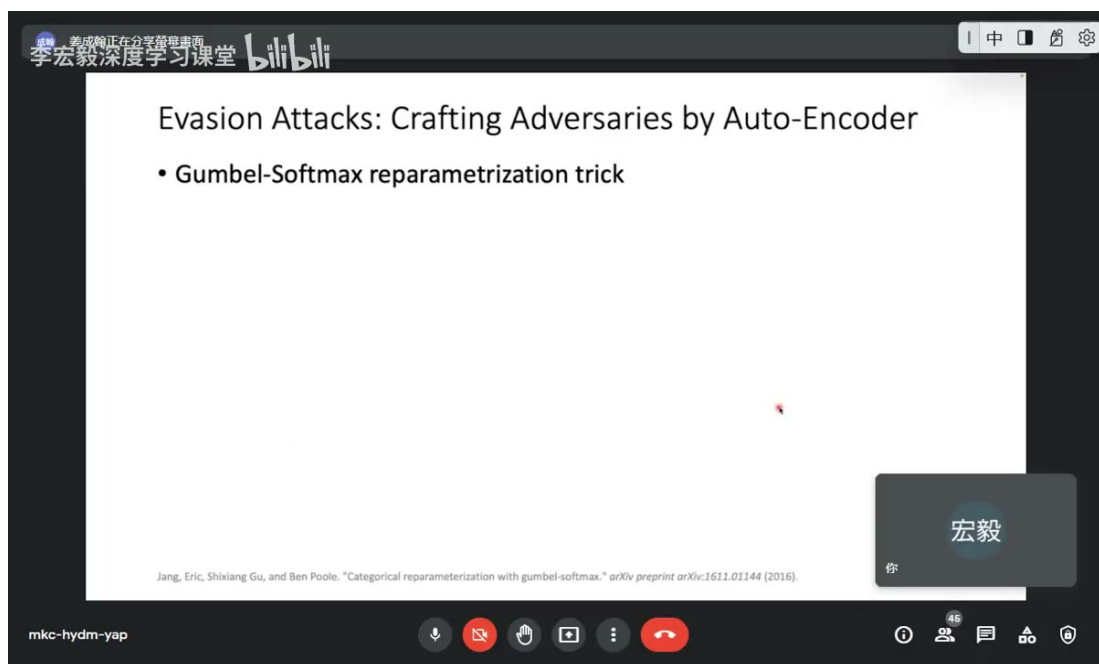


图 5: Gumbel-Softmax reparameterization trick 从 Gumbel-Max 出发，把离散采样改写成可近似微分的形式。⁵

Gumbel-Max 本身仍包含 $\arg \max$ ，所以还不可微；但它把随机性显式写成 g_i ，为后续的 softmax relaxation 铺路。

2.3 Softmax temperature relaxation

Gumbel-Softmax 用带温度 T 的 softmax 近似 $\arg \max$ ：

$$y_i = \frac{\exp((\log \pi_i + g_i)/T)}{\sum_{j=1}^V \exp((\log \pi_j + g_j)/T)}.$$

⁵视频画面时间区间：00:14:15–00:14:30。

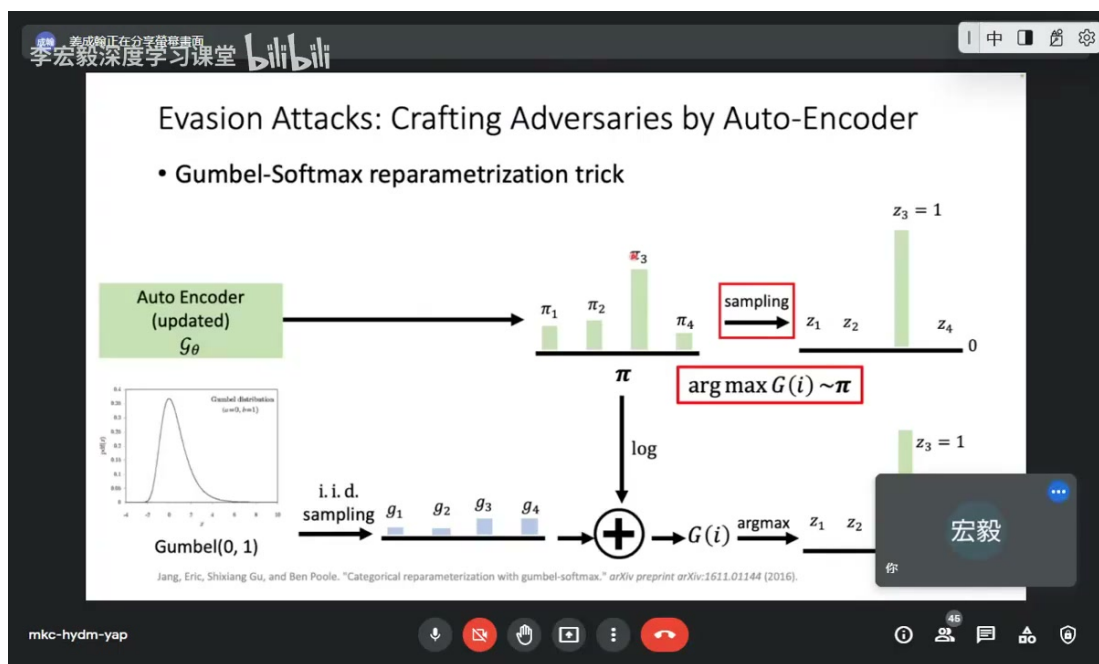


图 6: Gumbel-Softmax 用 softmax with temperature scaling 近似 argmax，使 categorical sampling 有连续替代。⁶

当 T 很小时，最大的那一项会接近 1，其余接近 0， y 接近 one-hot；当 T 较大时，分布更平滑，梯度更稳定但离散 token 更远。温度 T 因此控制了“可微性”和“离散性”的折中。

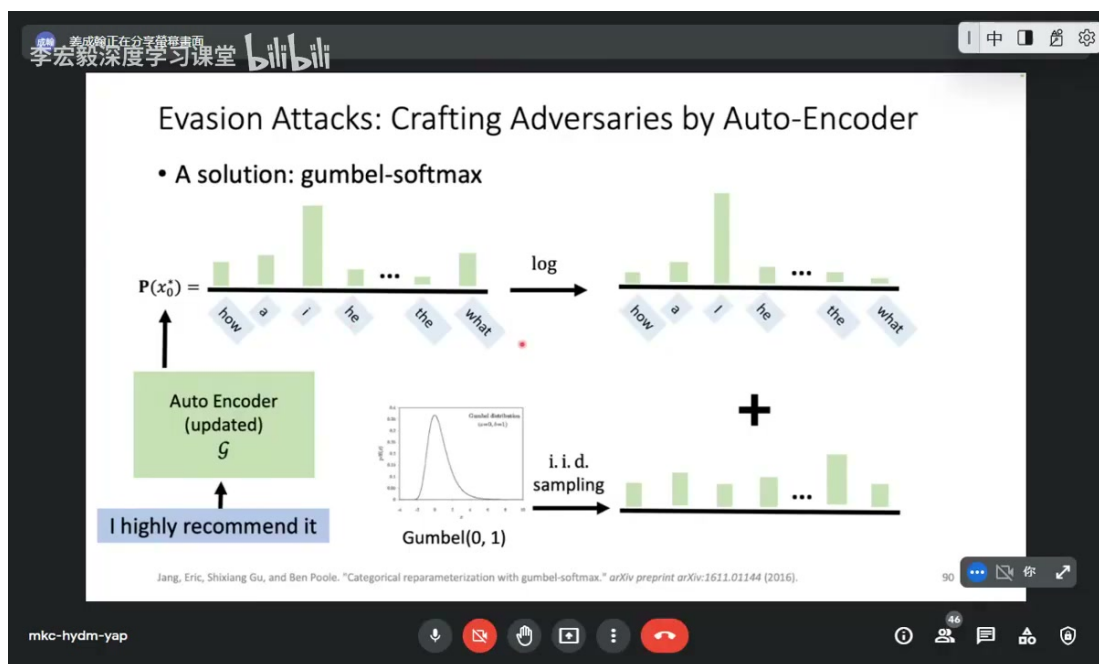


图 7: Gumbel-Softmax 把 generator 的词表分布、Gumbel noise 和 temperature softmax 组合成可训练路径。⁷

训练时可以把 y 当作 soft one-hot，再用 embedding matrix $E \in \mathbb{R}^{V \times d}$ 得到连续 embedding：

$$e = E^\top y.$$

⁶ 视频画面时间区间：00:19:45–00:20:00。

⁷ 视频画面时间区间：00:24:30–00:24:45。

这样 classifier 接收到的是所有词 embedding 的加权和，而不是单个离散词。虽然这不完全等同真实文本，但它允许梯度通过 e 、 y 回到生成器参数 θ 。

2.4 近似 one-hot 与梯度回传

课程强调，Gumbel-Softmax 最终不是直接生成一个真实 token，而是用一个接近 one-hot 的分布近似 token。这个近似分布可以送入 embedding table，形成可微的“软 token”。

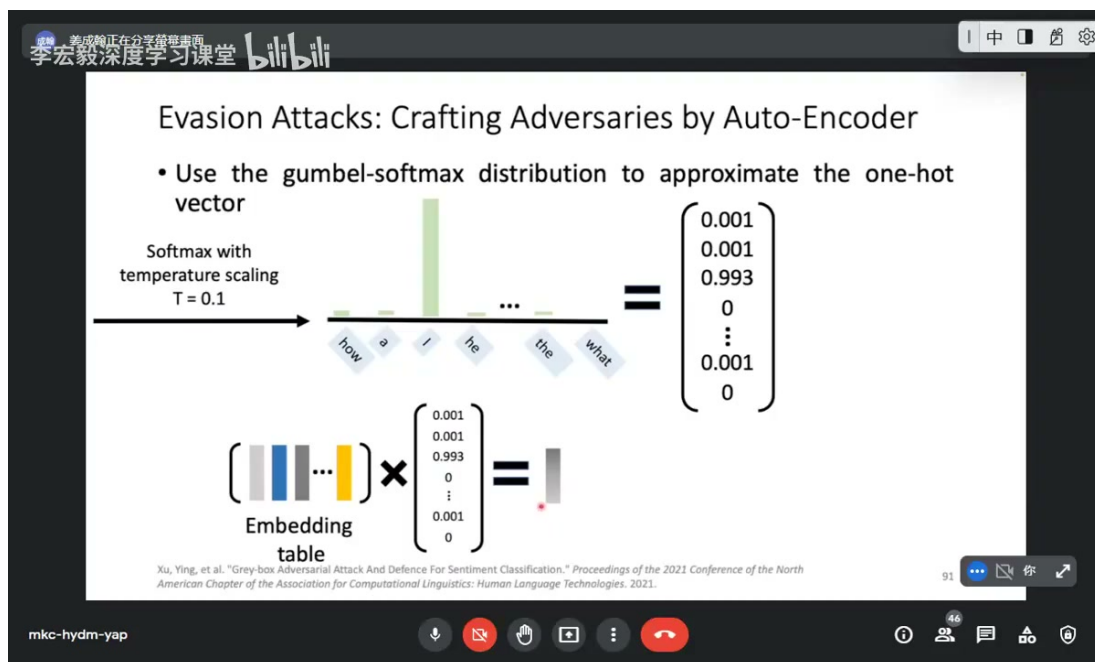


图 8: 使用 Gumbel-Softmax 分布近似 one-hot vector，再与 embedding table 相乘得到连续表示。⁸

因此，攻击训练可以形成完整反向路径：

$$G_{\theta}(x) \rightarrow \pi \rightarrow y_{\text{gumbel}} \rightarrow E^{\top} y_{\text{gumbel}} \rightarrow C(\cdot) \rightarrow L_{\text{adv}}.$$

⁸视频画面时间区间：00:25:45–00:26:00。

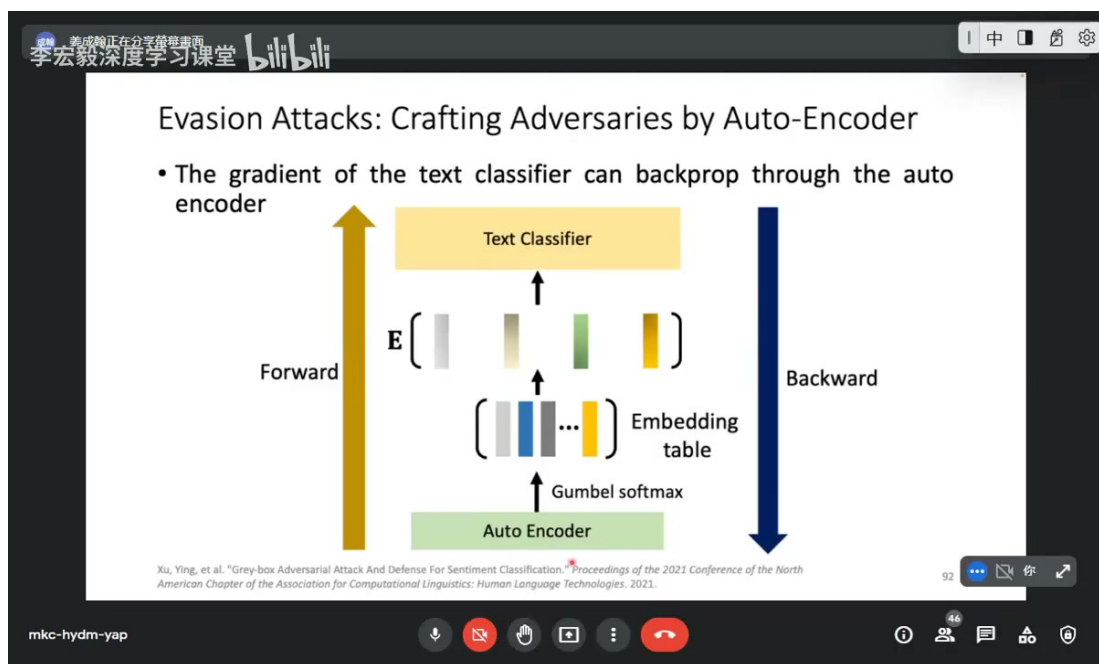


图 9: 由于 Gumbel-Softmax 提供连续近似，text classifier 的梯度可以回传穿过 auto-encoder。⁹

实际使用时常见两种变体。第一种是纯 soft relaxation：前向和反向都用连续 y 。第二种是 straight-through estimator：前向用 hard one-hot token，反向时假装它是 soft y ，让梯度穿过。前者训练更平滑，后者与真实离散输入更接近，但梯度估计偏差更明显。

Gumbel-Softmax 解决的不是“语义”问题

Gumbel-Softmax 只解决离散采样不可微的问题。它让生成器能从分类器 loss 得到梯度，但不自动保证生成句语义正确。语义保持仍要靠 reconstruction、similarity、POS、perplexity 或人工评估等约束。

2.5 本章小结

文本生成式攻击的关键障碍是 token 采样不可微。Gumbel-Softmax 用 Gumbel noise 加 temperature softmax 近似 categorical sampling，使分类器梯度可以穿过 soft token 回到 generator。它把离散文本攻击暂时搬到连续空间中训练，再用近似 one-hot 的方式靠近真实 token。

3 训练一个执行扰动的 Agent

除了用 auto-encoder 直接生成整句，还可以把攻击看成一个 agent 的决策过程。Agent 读入句子，决定在哪些位置做什么扰动；分类器反馈当前扰动是否有效；训练目标是让 agent 学会用少量、合理的动作降低正确类别概率。

⁹ 视频画面时间区间：00:27:15–00:27:30。

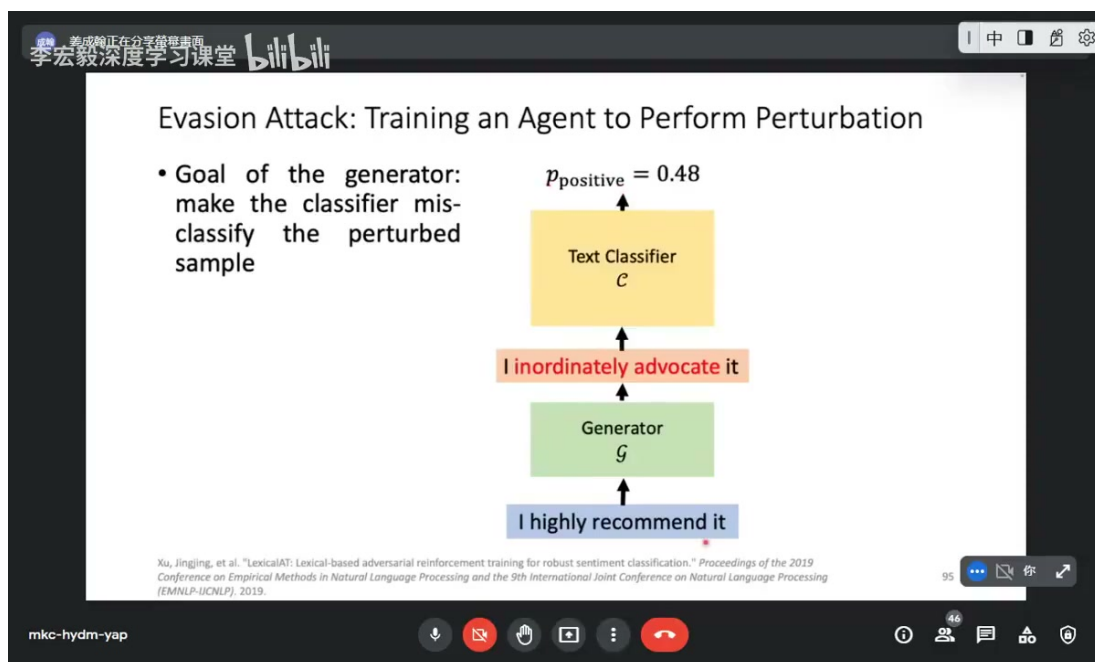


图 10: Perturbation agent 的目标：生成扰动样本，使 text classifier 对原类别的判断下降甚至翻转。¹⁰

3.1 动作空间

Agent 的动作可以设计成不同粒度。简单版本只在每个词上决定“保持不变”或“替换成某个候选词”；复杂版本可以允许替换、插入、删除、同义改写、形态变化等动作。课程中的表格展示的是对词进行替换类操作的动作集合。

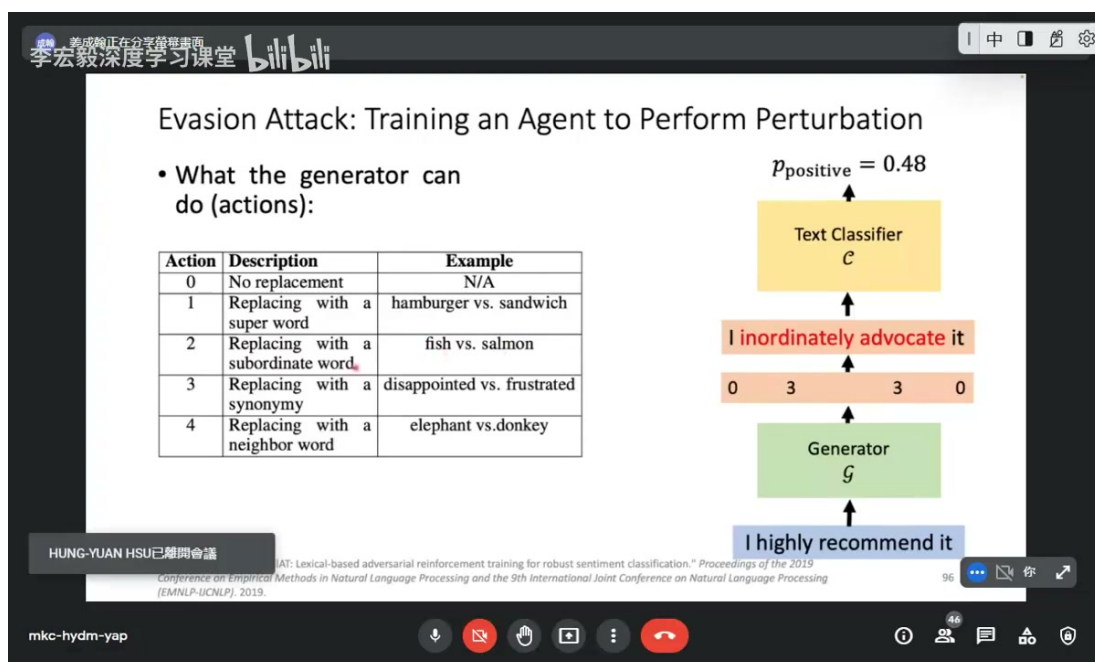


图 11: Agent 的 action space 描述它能对文本做哪些扰动，例如保留当前词或替换成候选词。¹¹

¹⁰ 视频画面时间区间：00:30:00–00:30:15。

¹¹ 视频画面时间区间：00:31:45–00:32:00。

可以把决策写成策略：

$$a_t \sim \pi_\theta(a_t | s_t),$$

其中 s_t 是当前句子状态， a_t 是扰动动作。执行动作后得到新状态 s_{t+1} ，也就是新的改写句子。

动作空间设计会影响攻击质量。动作太自由，容易产生不自然句子；动作太保守，攻击成功率可能很低。因此常见做法是先用同义词、词向量邻居或语言模型候选限制动作集合，再让 agent 在这些候选里选择。

3.2 强化学习奖励

因为动作是离散选择，强化学习是自然选择。课程用分类器对 ground truth class 的概率下降作为 reward：如果扰动后正确类别概率下降，agent 得到正奖励；如果概率没有下降或样本质量变差，则奖励较小或为负。

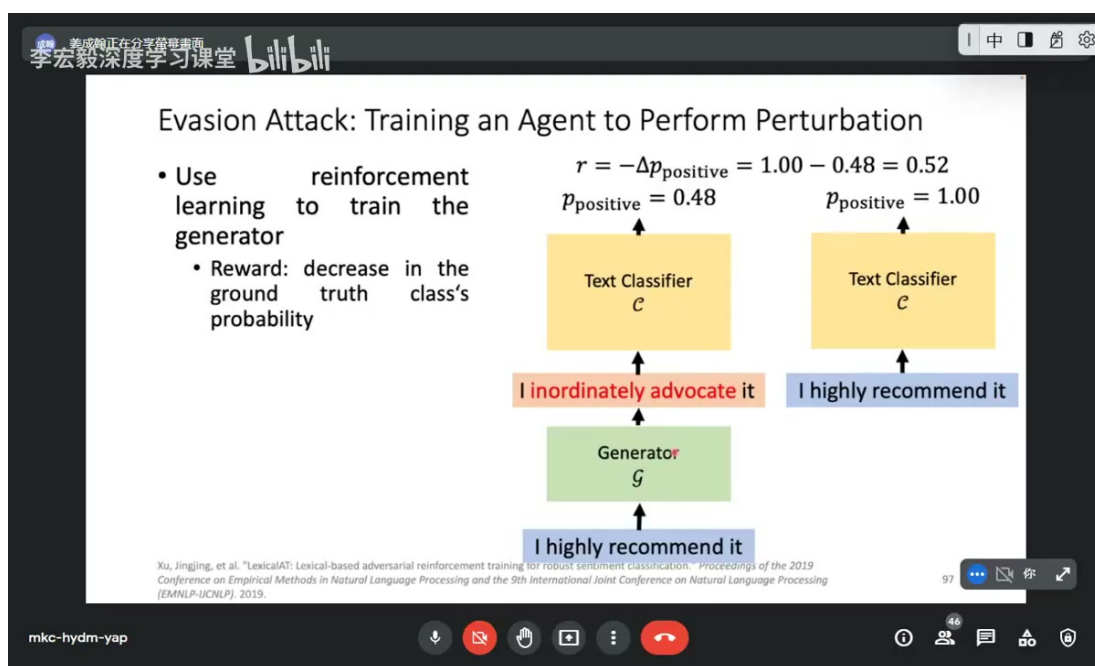


图 12: 强化学习奖励可以定义为真实类别概率的下降量，例如 $r = -\Delta p_y$ 。¹²

若当前句子为 x_t ，执行动作后为 x_{t+1} ，可以定义：

$$r_t = p_C(y | x_t) - p_C(y | x_{t+1}).$$

若 $r_t > 0$ ，表示动作降低了模型对正确标签的信心；若 $r_t < 0$ ，表示动作反而让模型更确定。对 targeted attack，也可以把 reward 改成目标类别概率的上升量：

$$r_t = p_C(y_{\text{tar}} | x_{t+1}) - p_C(y_{\text{tar}} | x_t).$$

训练时可用 policy gradient：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_t R_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right],$$

其中 R_t 是从当前步骤开始的累积奖励。

¹²视频画面时间区间：00:33:45–00:34:00。

3.3 与 greedy 搜索的关系

Perturbation agent 和 greedy/WIR 的目标相似：都是找一串离散替换动作。不同之处在于 greedy 是手写规则，每一步选当前最优；agent 是从经验中学策略，可以把多步效果纳入考虑。理论上，agent 可以学到“某个动作单看不强，但为后续动作铺路”的策略。

不过，强化学习训练也更不稳定，且需要大量 interaction。若分类器查询昂贵，或奖励设计不够精细，agent 可能学到低质量扰动。因此实际评估仍要看修改比例、语义相似度、语法自然度和查询成本。

3.4 本章小结

Perturbation agent 把文本攻击写成序列决策问题。动作空间决定 agent 能改什么，reward 决定 agent 追求什么。用真实类别概率下降作为奖励，可以训练 agent 生成更容易欺骗分类器的文本；但离散动作和样本质量约束仍是核心难题。

4 防御一：Adversarial Training

课程转入 defense 后，首先讲的是最直接的策略：在训练过程中加入 adversarial examples，让模型见过这些扰动模式。图像领域已有类似思想；在文本领域，困难在于 adversarial samples 的生成更贵，而且文本扰动必须保持语义。

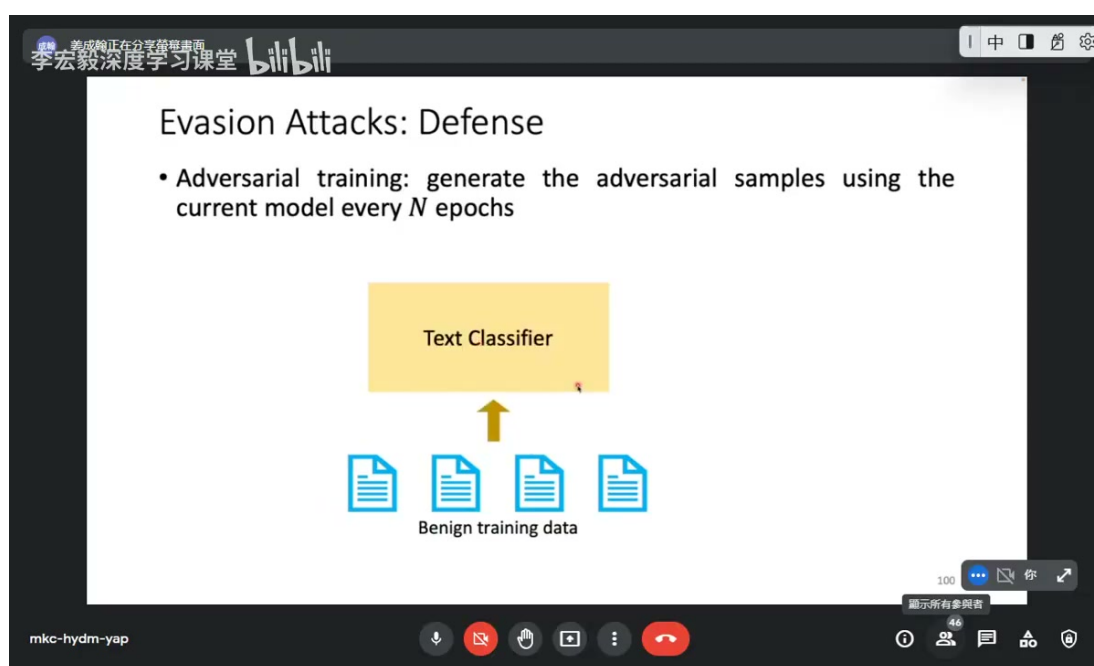


图 13: Adversarial training 可以定期用当前模型生成 adversarial samples，再把它们加入训练。¹³

4.1 标准训练流程

标准 adversarial training 可写成：

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\max_{x' \in \mathcal{A}(x)} L(C_{\theta}(x'), y) \right].$$

¹³ 视频画面时间区间：00:35:45–00:36:00。

其中 $\mathcal{A}(x)$ 是允许的扰动集合，包括修改比例、语义相似、POS consistency 等约束。内层最大化找最能让模型出错的扰动，外层最小化让模型在这些扰动上仍然正确。

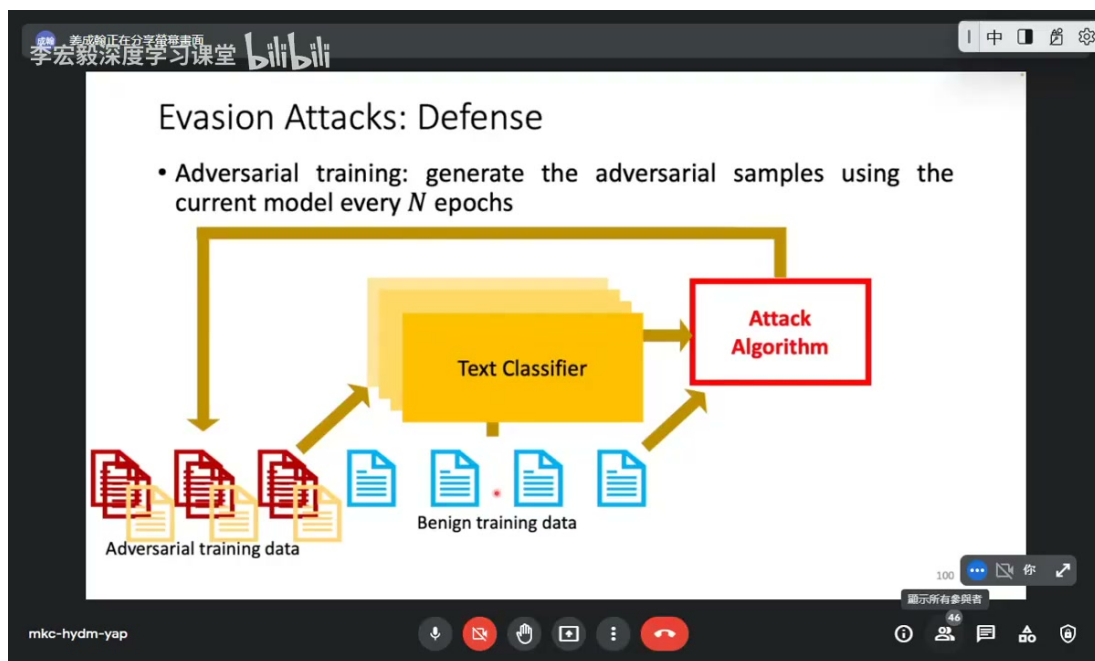


图 14: 训练循环：攻击算法生成 adversarial training data，模型再用这些样本更新参数。¹⁴

实际训练中，不可能每一步都精确求解内层最大化，通常用已有攻击算法近似生成样本。例如每若干 epoch 用当前模型生成一批 adversarial examples，加入训练集，继续训练模型。这样模型会逐渐适应当前攻击器能找到的弱点。

4.2 文本 adversarial training 的局限

文本 adversarial training 的主要限制有三点：

1. **成本高**：生成文本 adversarial examples 往往需要大量模型查询、候选过滤和语义检查。
2. **攻击器依赖**：模型可能只对训练时使用的攻击方法鲁棒，对新攻击方法仍脆弱。
3. **样本质量难控**：若加入低质量 adversarial samples，模型可能学到错误标签或偏离真实任务。

因此，adversarial training 不是万能防御。它更像是一种压力训练：把已知攻击样本加入训练，让模型在这些局部区域变得更平滑、更稳定。

4.3 本章小结

Adversarial training 的思想简单但成本不低：先用攻击器生成难样本，再用这些样本训练鲁棒模型。对 NLP 来说，它必须建立在高质量攻击样本之上，否则防御可能只是记住低质量改写，而不是真正提升语义鲁棒性。

5 防御二：Embedding 空间与 ASCC

直接在离散文本上找 adversarial examples 很难，防御者也可以转向 embedding space。一个词的同义词往往在 embedding 空间中相近；若模型在这个邻域内都能保持预测稳定，就可能对同义替换攻击更鲁棒。

¹⁴ 视频画面时间区间：00:37:15–00:37:30。

5.1 Embedding ball adversarial training

Embedding ball 方法把每个词的 embedding e_i 看成连续向量，在其附近找扰动：

$$e'_i = e_i + \delta_i, \quad \|\delta_i\| \leq \epsilon.$$

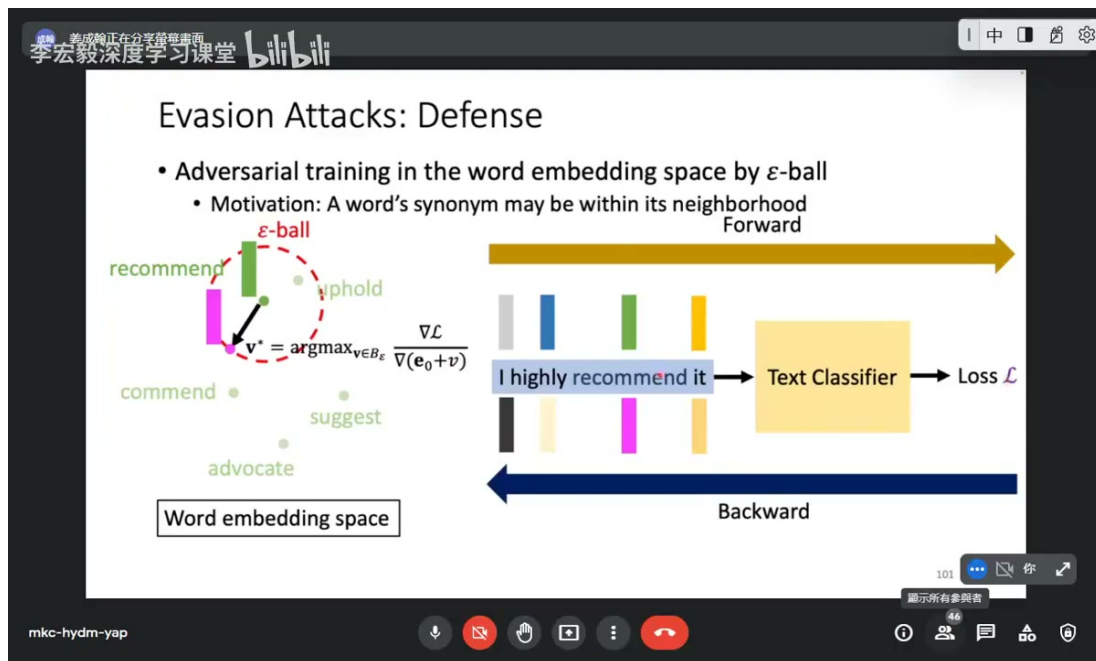


图 15: Embedding ball adversarial training 在词向量邻域中寻找扰动，并让模型对邻域内输入保持正确。¹⁵

训练目标类似：

$$\min_{\theta} \mathbb{E}_{(x,y)} \left[\max_{\|\delta_i\| \leq \epsilon} L(C_{\theta}(e_1 + \delta_1, \dots, e_n + \delta_n), y) \right].$$

优点是连续优化方便，可以用梯度方法找到 worst-case perturbation；缺点是 embedding ball 中的点未必对应真实词。模型在连续空间里变鲁棒，不一定等价于对真实同义词替换鲁棒。

5.2 ASCC 的动机

ASCC (Adversarial Sparse Convex Combination) 试图解决这个问题。与其允许任意 ϵ -ball 里的连续点，不如只考虑某个词的同义词集合构成的 convex hull。这样 adversarial embedding 仍是连续组合，但组合基底来自真实候选词。

¹⁵ 视频画面时间区间：00:39:45–00:40:00。

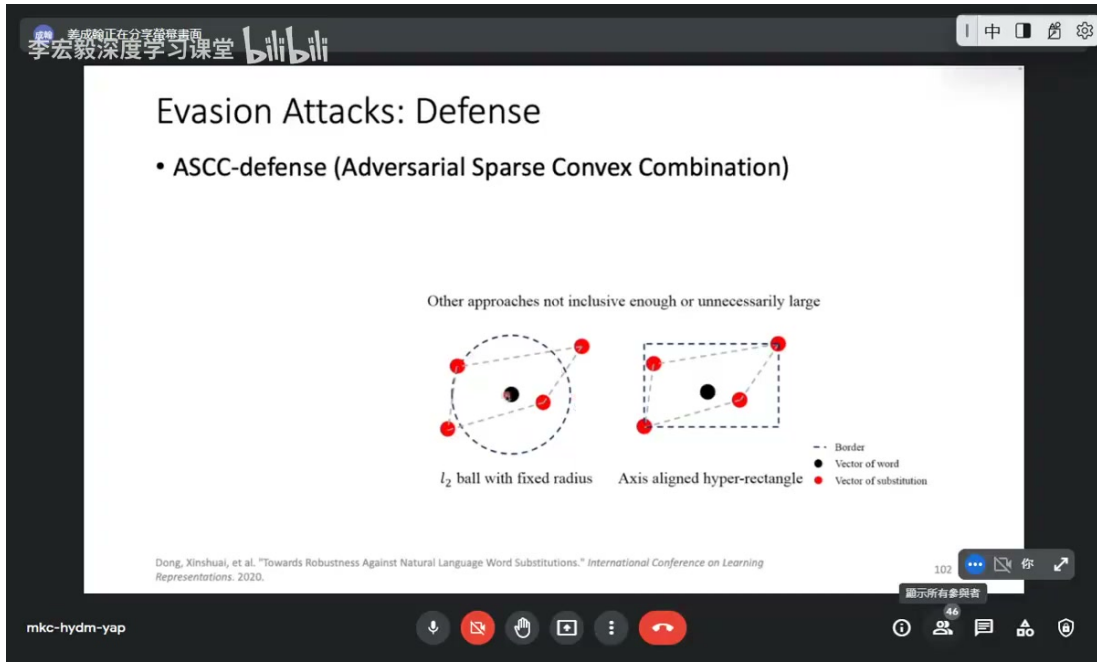


图 16: ASCC 的基本想法: 用同义词集合的 convex hull 作为扰动空间, 而不是任意 embedding ball。¹⁶

设词 w_i 的候选集合为:

$$S_i = \{w_{i1}, w_{i2}, \dots, w_{ik}\},$$

其 embedding 为 $e(w_{ij})$ 。ASCC 构造:

$$\tilde{e}_i = \sum_{j=1}^k \alpha_{ij} e(w_{ij}), \quad \alpha_{ij} \geq 0, \quad \sum_{j=1}^k \alpha_{ij} = 1.$$

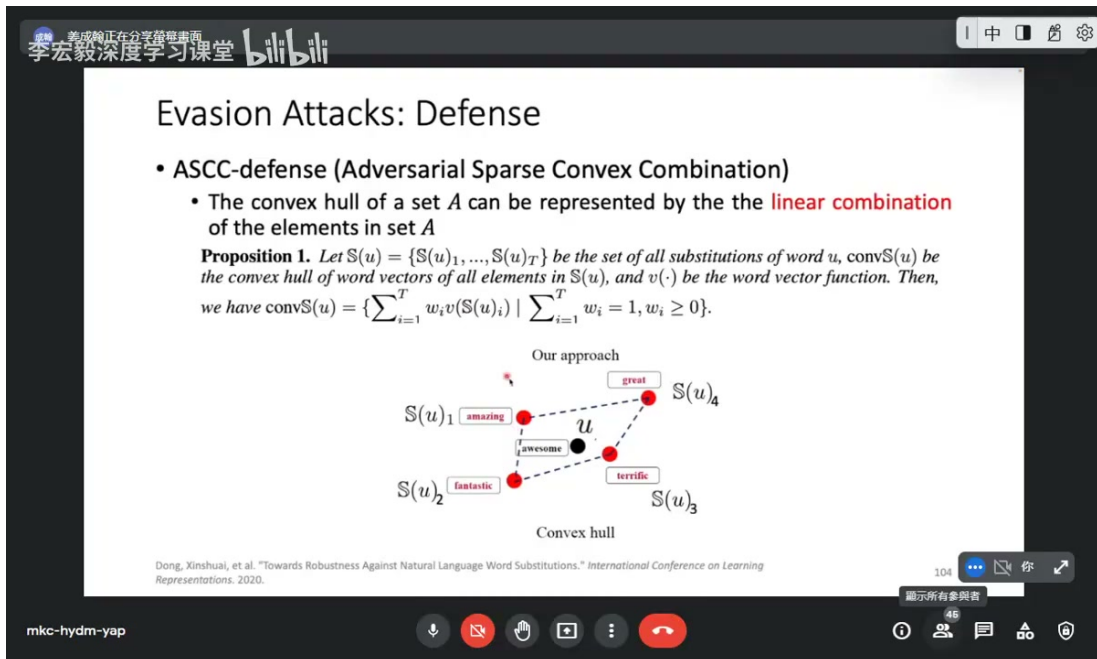


图 17: ASCC 用候选同义词 embedding 的线性凸组合表示 adversarial embedding。¹⁷

¹⁶ 视频画面时间区间: 00:42:00–00:42:15。

这让扰动空间从“所有邻域点”变成“同义词候选张成的凸包”。如果候选集合质量较高，扰动就更接近可解释的词替换。

5.3 稀疏性与最坏组合

ASCC 还希望线性组合不要平均混合太多词，而是更接近少数候选词。因此目标中会加入鼓励 sparse coefficient 的项。一个概念化目标为：

$$\max_{\alpha} L(C_{\theta}(\tilde{e}_1, \dots, \tilde{e}_n), y) - \lambda \sum_i H(\alpha_i),$$

其中 $H(\alpha_i)$ 是位置 i 上组合系数的熵。最大化时惩罚高熵分布，会使 α_i 更尖锐，接近选择少数候选词。

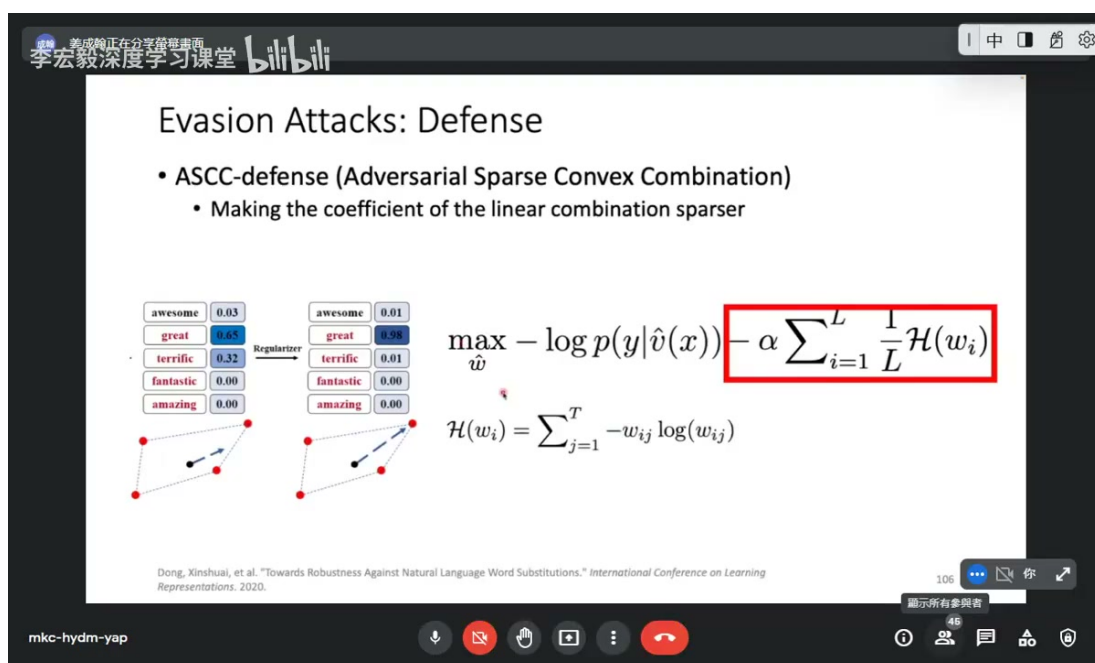


图 18: ASCC 在最大化分类损失的同时约束线性组合系数，使 adversarial embedding 更接近稀疏候选组合。¹⁸

随后模型在这些 worst-case convex combinations 上训练：

$$\min_{\theta} \mathbb{E}_{(x,y)} [L(C_{\theta}(\tilde{e}^*), y)], \quad \tilde{e}^* = \arg \max_{\tilde{e} \in \text{Conv}(S)} L(C_{\theta}(\tilde{e}), y).$$

ASCC 的优势是比任意 embedding ball 更贴近语义候选；局限是它高度依赖候选同义词集合。如果候选集合本身包含语义不等价的词，防御训练也会受到污染。

5.4 用不鲁棒分类器做数据增强

课程还提到一种数据增强思路：先训练一个不够鲁棒的分类器，用攻击算法从它身上生成 adversarial samples，再把这些样本加入训练数据，训练一个新的分类器。

¹⁷ 视频画面时间区间：00:45:45–00:46:00。

¹⁸ 视频画面时间区间：00:49:30–00:49:45。

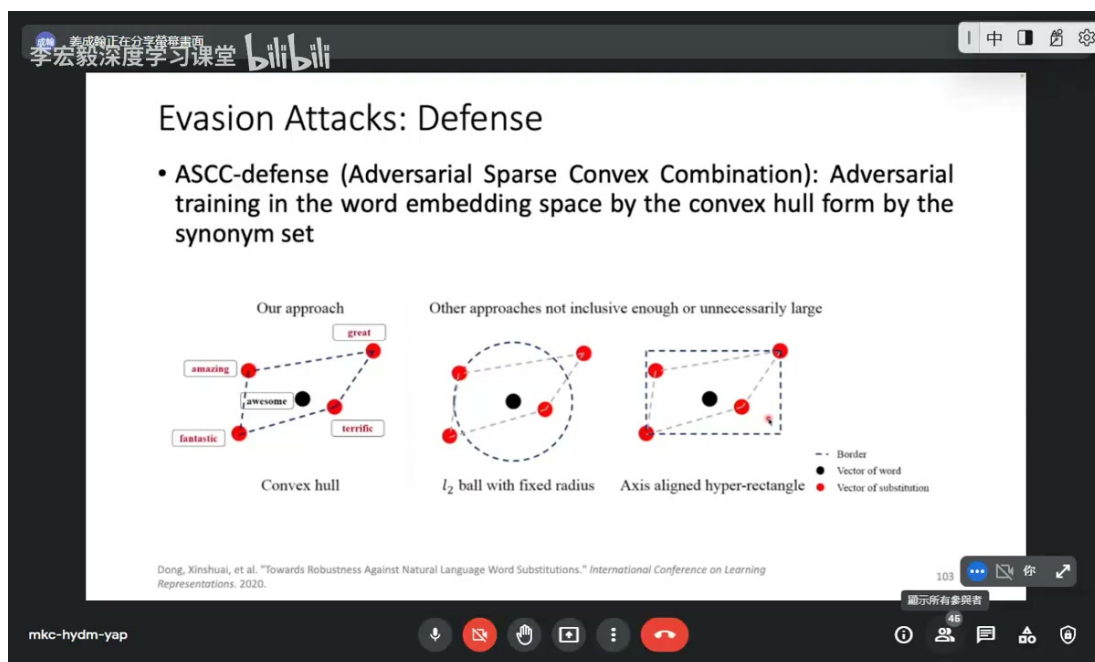


图 19: 也可以先用不鲁棒分类器预生成 adversarial samples，再把它们加入训练集训练新模型。¹⁹

这种方法的优点是流程简单，且不需要在每次训练迭代中都运行昂贵攻击；缺点是生成样本只覆盖旧模型的脆弱点，新模型仍可能被其他攻击方式击中。

连续鲁棒性与离散鲁棒性

Embedding-space defense 让训练更容易优化，但文本最终仍是离散 token。若连续扰动空间和真实可替换词集合不匹配，模型可能在数学邻域里鲁棒，却在真实同义词攻击下仍不稳定。ASCC 的价值就在于把连续优化限制到同义词凸包附近。

5.5 本章小结

Embedding ball adversarial training 把文本防御搬到连续向量空间中，方便使用梯度寻找最坏扰动；ASCC 进一步把扰动限制在同义词候选的 convex hull 内，让连续扰动更接近真实词替换。二者都试图提升模型对语义相近改写的稳定性，但都依赖候选集合和语义约束的质量。

6 防御三：推理期检测与修复

除了训练更鲁棒的模型，也可以在 inference time 检测输入是否被攻击，并尝试把它修复回原句。课程介绍的代表方法是 DISP，以及一个基于词频的简单检测/替换思路 FGWS。

¹⁹ 视频画面时间区间：00:52:15–00:52:30。

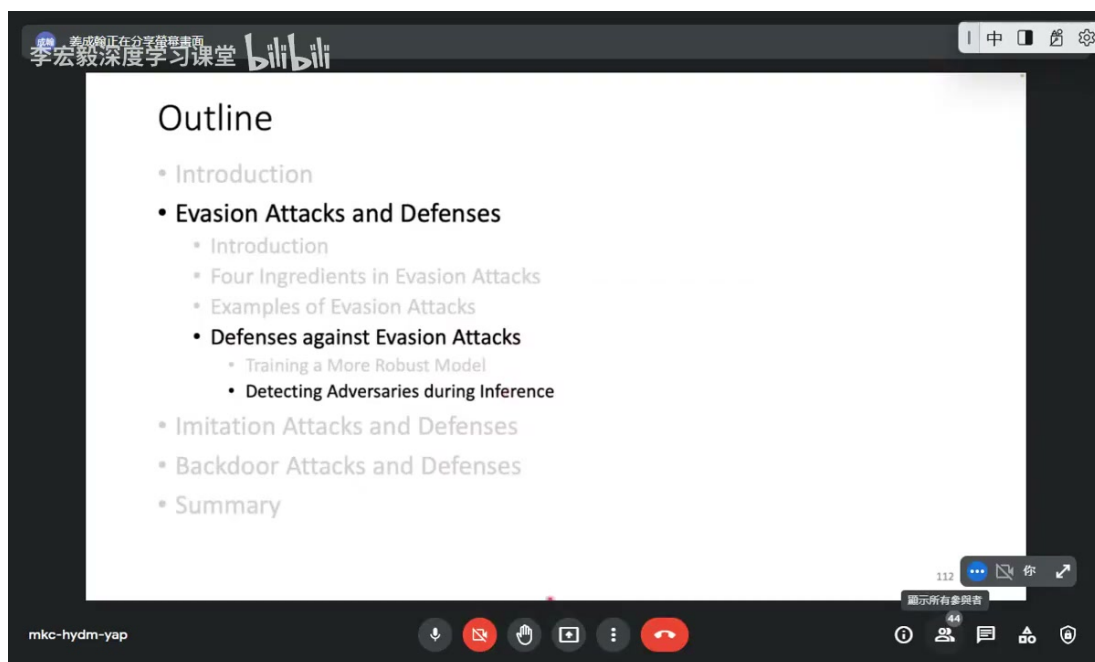


图 20: 推理期防御关注 detecting adversaries during inference: 输入进来后先判断是否含扰动, 再尝试修复。²⁰

6.1 DISP: 三段式检测与恢复

DISP (Discriminate Perturbations) 把问题拆成三个子模块:

1. **Perturbation discriminator**: 判断每个 token 是否被扰动。
2. **Embedding estimator**: 估计被扰动位置原本应该对应的 embedding。
3. **Token recovery**: 在 embedding corpus 中找最近词, 把可疑 token 替换回去。

²⁰ 视频画面时间区间: 00:54:45–00:55:00。

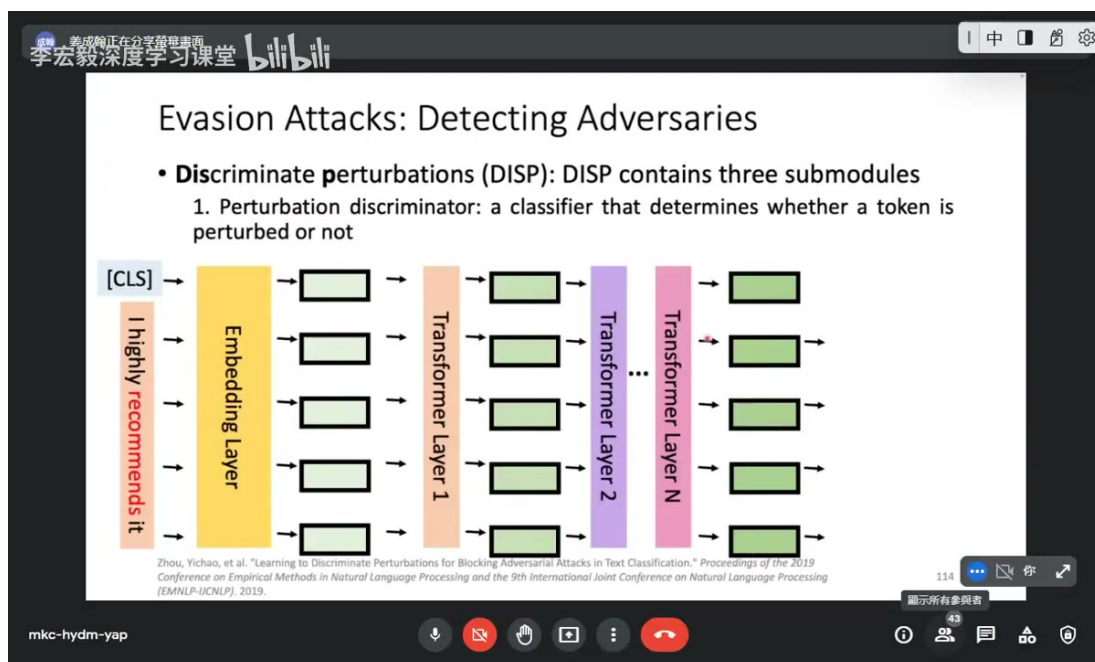


图 21: DISC 包含 perturbation discriminator、embedding estimator 和 token recovery 三个子模块。²¹

这种设计的直觉是：对抗样本中的替换词虽然在表面上像同义词，但在上下文中可能有异常痕迹。若能定位异常 token，再估计原始 token 的 embedding，就可以把句子修复到更接近 clean input 的状态。

6.2 Embedding estimator

Embedding estimator 的任务不是直接猜原词，而是先预测原词 embedding。它可以利用上下文表示、双向模型或邻近 token 信息，输出一个连续向量 $\hat{e}_i$ ：

$$\hat{e}_i = R_\psi(h_i),$$

其中 h_i 是位置 i 的上下文表示， R_ψ 是 regression head。

²¹ 视频画面时间区间：00:56:00–00:56:15。

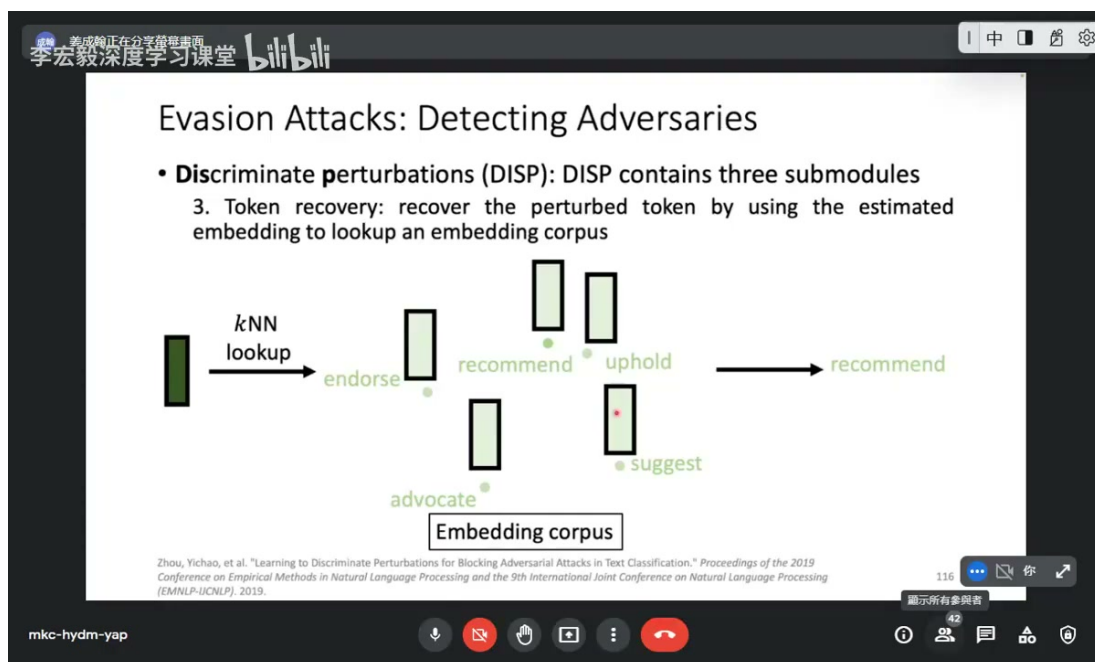


图 22: Embedding estimator 利用上下文估计被扰动 token 原本的 embedding, 再与词表 embedding 比较。²²

随后 token recovery 可以执行最近邻检索:

$$\hat{w}_i = \arg \min_{w \in \mathcal{V}} \|\hat{e}_i - e(w)\|_2.$$

若 perturbation discriminator 认为某个位置没有被扰动, 就不需要替换; 若认为它被扰动, 则用 $\hat{w}_i$ 替换当前 token。

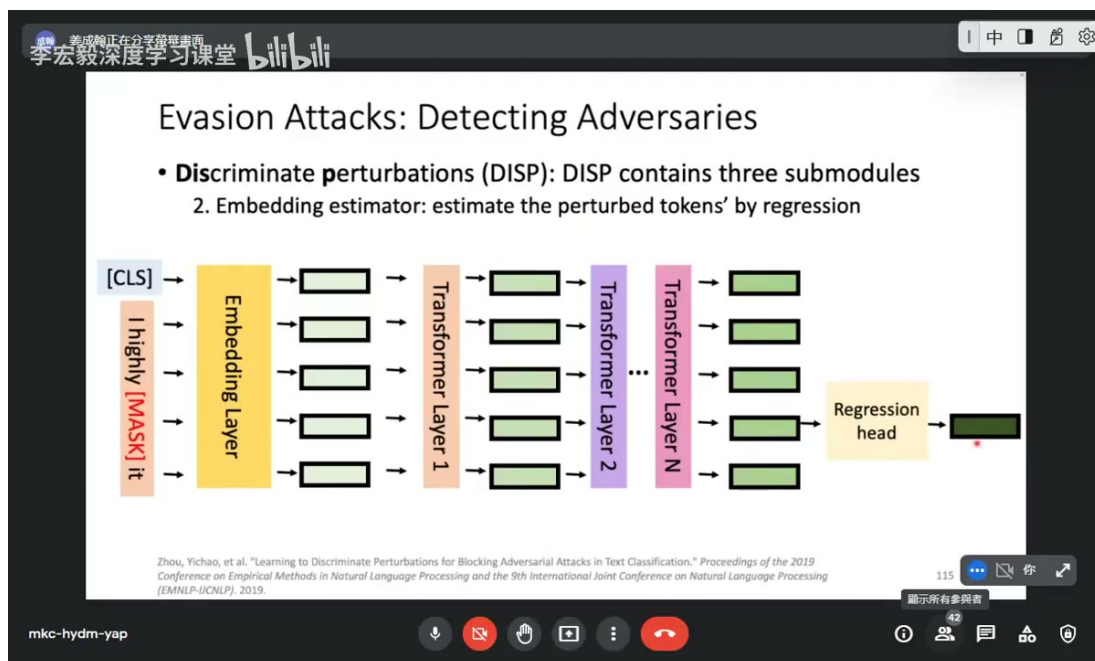


图 23: DISP 中的 regression head 负责把上下文表示映射回估计的原始 embedding。²³

²² 视频画面时间区间: 00:58:30-00:58:45。

²³ 视频画面时间区间: 01:01:00-01:01:15。

6.3 训练与推理流程

DISP 需要构造 clean sentence 与 perturbed sentence 的配对数据。训练时，系统知道哪些 token 被替换，因此可以监督 perturbation discriminator；同时也知道原词 embedding，因此可以训练 embedding estimator。

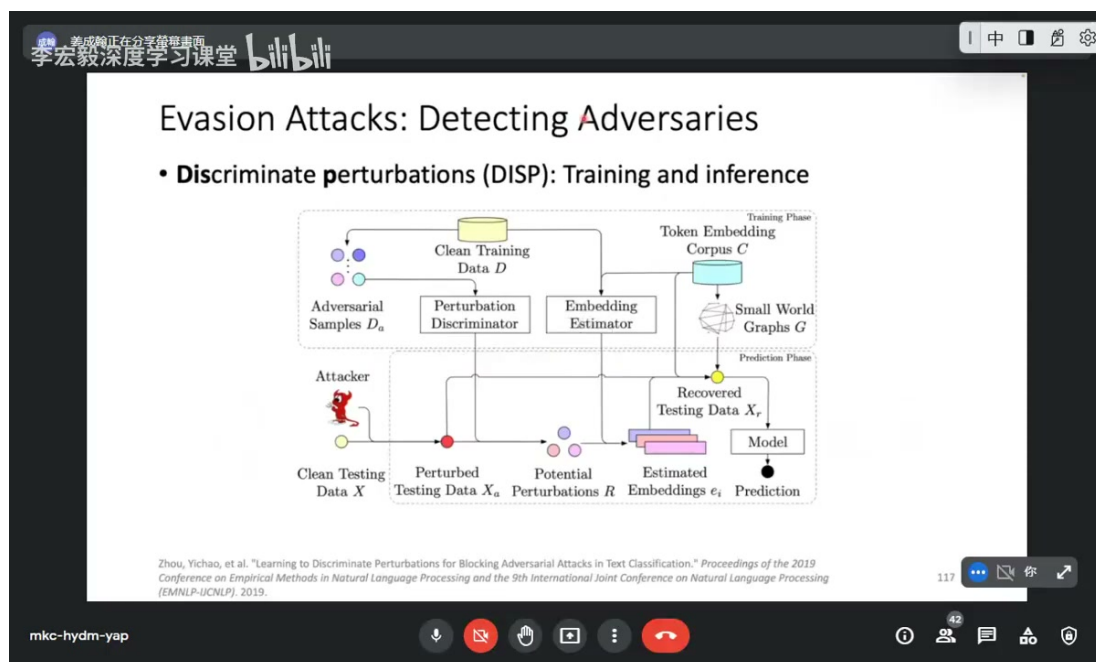


图 24: DISP 训练时使用 clean/perturbed 配对监督各模块，推理时检测并恢复可疑扰动。²⁴

推理时流程是：

1. 输入一个可能被攻击的句子。
2. discriminator 标出可疑 token。
3. estimator 对可疑位置估计原始 embedding。
4. recovery 从词表中找最近 token 进行替换。
5. 将修复后的句子送入原分类器。

DISP 的优点是能在模型预测前做预处理；局限是它假设扰动具有可检测模式。如果攻击者知道防御器存在，可能生成更像自然输入的替换，绕过 detector。

6.4 FGWS：用词频变化发现可疑替换

FGWS (Frequency-Guided Word Substitution) 利用一个经验观察：许多文本攻击会把常见词替换成语义相近但频率较低的词，从而骗过模型。于是可以尝试把低频可疑词替换成更高频的对应词，再比较模型输出变化。

²⁴ 视频画面时间区间：01:03:30–01:03:45。

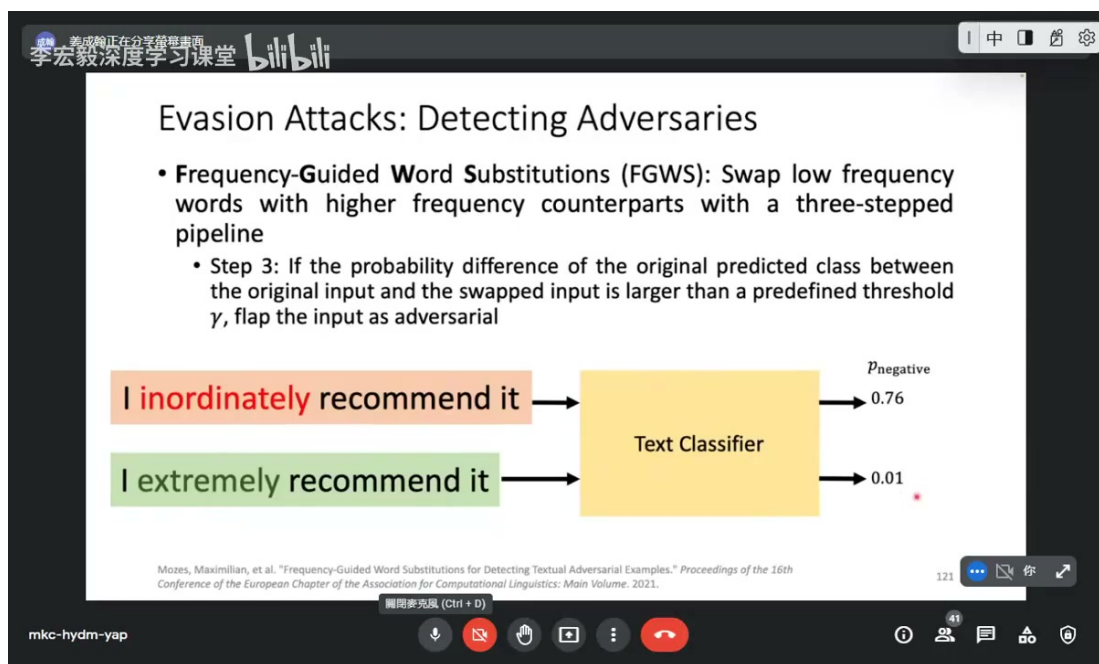


图 25: FGWS 比较原输入与替换后输入在模型预测概率上的差异；若差异超过阈值，就标记为 adversarial。²⁵

一个简化检测规则为：

$$\Delta p = p_C(\hat{y} \mid \text{swap}(x)) - p_C(\hat{y} \mid x),$$

其中 $\hat{y} = C(x)$ 是原输入的预测类别， $\text{swap}(x)$ 表示把低频词替换成高频候选后的句子。若：

$$\Delta p > \gamma,$$

则认为原输入可能是 adversarial input。

FGWS 的优点是简单，不需要复杂训练；缺点是启发式很强。它依赖“攻击倾向使用低频词”这个假设，对使用高频自然替换、上下文语言模型替换或语义更隐蔽的攻击可能不够有效。

检测式防御的共同风险

检测器本身也会成为攻击目标。若攻击者知道检测规则，就可以优化样本同时欺骗分类器和检测器。因此推理期检测适合作为多层防御的一部分，不应被视为单独充分的安全保证。

6.5 本章小结

推理期防御不直接改变分类器，而是在输入进入分类器前检测和修复。DISP 通过 token-level detector、embedding estimator 和 nearest-neighbor recovery 还原可疑扰动；FGWS 通过低频词替换与概率差阈值做启发式检测。它们能补充 adversarial training，但都面临适应性攻击的挑战。

7 总结与延伸

P42 把 NLP evasion attack 的后半部分补完整：前半讲关注如何生成攻击样本，后半讲关注如何防御或检测攻击样本。整讲的主线是“离散文本如何被连续优化处理”。

在攻击侧，auto-encoder attack 把 adversarial example generation 写成训练目标：重建 loss 保持句子接近，similarity loss 保持语义接近，adversarial loss 推动分类器犯错。由于 token sampling

²⁵ 视频画面时间区间：01:07:30–01:07:45。

不可微，Gumbel-Softmax 用 soft one-hot 近似离散采样，让分类器梯度可以穿过 generator。另一条路线是训练 perturbation agent，把文本改写看成离散动作序列，用真实类别概率下降作为 reward。

在防御侧，adversarial training 让模型见过攻击样本；embedding-space training 把离散替换放到连续空间中寻找最坏扰动；ASCC 则进一步把扰动限制在同义词 convex hull 中，试图兼顾可优化性和语义合理性。推理期方法如 DISP 和 FGWS 不重新训练主模型，而是在输入侧检测、修复或替换可疑 token。

这几类方法共同说明，NLP 对抗攻防的本质不是单个算法名，而是三个问题的组合：

- **优化问题**：离散 token 如何接受梯度或搜索？
- **语义问题**：改写后是否仍是同一个输入？
- **安全问题**：防御是否只挡住已知攻击，还是能应对自适应攻击者？

从实践角度看，单一防御很难覆盖所有攻击。较可靠的系统通常需要组合：训练时使用高质量 adversarial examples 增强鲁棒性，模型设计时检查 embedding 与同义词邻域的稳定性，推理时加入检测和异常监控，评估时用多种攻击方法与人工语义检查验证。NLP 模型越接近真实应用，越不能只看 clean accuracy；它还必须经受“语义不变但表面改变”的压力测试。